

# [4] Accelerating Machine Learning Inference with Probabilistic Predicates 총정리

**저자:** Yao Lu, Aakanksha Chowdhery, Srikanth Kandula, Surajit Chaudhuri (Microsoft Research) **학회/년도:** SIGMOD 2018 (ACM International Conference on Management of Data) **분량:** 약 16페이지 **역할군:** (D) 이론적 기반; ML+DB 최적화의 다른 각도

## 요약

이 논문은 "비구조화 데이터에 대한 ML 추론 쿼리에서 비싼 UDF(User-Defined Function)를 어떻게 최적화할 것인가"라는 문제를 다룬다. Exqutor와는 다른 각도에서 ML+DB 최적화를 접근하지만, 두 논문 모두 "ML/벡터 연산을 관계형 쿼리 옵티마이저에 어떻게 통합할 것인가"라는 큰 질문을 공유한다.

핵심 아이디어는 비싼 ML 모델(딥러닝 기반 객체 탐지, 얼굴 인식 등) 이전에 **경량 이진 분류기**를 "조기 필터"로 삽입하는 것이다. 이를 통해 불필요한 데이터를 미리 걸러내고 비싼 ML 추론의 횟수를 크게 줄인다. 여러 필터를 계단식으로 연결(cascade)하면 각 단계에서 점진적으로 데이터가 정제되어 전체 비용을 극적으로 감소시킨다.

Exqutor가 벡터 검색의 **카디널리티**를 정확히 추정하여 옵티마이저를 돕는 것과 유사하게, 이 논문은 ML 추론의 **비용과 선택도**를 옵티마이저에 통합한다. 두 논문의 공통 메시지는 "ML 연산의 특성을 옵티마이저에 정확히 반영하면 극적인 성능 향상이 가능하다"는 것이다.

본 논문과의 관계; 비싼 연산(벡터 검색 vs. ML 추론)의 특성을 옵티마이저에 통합하는 서로 다른 접근법을 보여주며, 두 방식의 결합 가능성을 시사한다.

## 상세분석

### 4.1 해결하고자 하는 문제

비디오 분석, 이미지 검색 같은 ML 추론 쿼리에서 핵심 병목은 **비싼 UDF(User-Defined Function)**이다.

## 문제 상황의 구체적 예시

```
SELECT * FROM video_frames
WHERE object_detector(frame, 'red SUV') = TRUE -- 매우 비쌘!
AND timestamp BETWEEN '2024-01-01' AND '2024-01-31';
```

여기서 `object_detector()` 는 딥러닝 모델로: - ResNet 또는 YOLO 같은 대규모 신경망 - 한 프레임당 수십~수백 ms가 소요됨 (GPU 사용 기준) - 100만 프레임 비디오라면;  $100만 \times 100ms = 100,000초 = \text{약 } 28\text{시간}$  소요

그런데 실제로 "빨간 SUV"가 나오는 프레임은 전체의 **0.1%도 안 될 수 있다** (예; 차가 10,000개 장면 중 10장에만 나타남).

따라서 불필요한 추론이 많아서, 이를 줄일 수 있다면 극적인 성능 개선이 가능하다.

## 기존 최적화의 한계

기존 쿼리 최적화의 **술어 푸시다운(Predicate Pushdown)** 기법:

```
[100만 프레임]
↓ [시간 필터 적용] (timestamp 조건)
↓ [31만 프레임] (1월 데이터만)
↓ [ML 추론 UDF 실행]
↓ [300 프레임] (빨간 SUV 있는 것)
↓ [결과 반환]
```

이것만으로도 도움이 되지만, **여전히 31만 프레임마다 비싼 DL 모델 실행**해야 한다. 추가로: -  $31만 \times 100ms = 31,000초 = \text{약 } 8.6\text{시간}$  (여전히 너무 김)

원근법을 바꿔서, **ML UDF 이전에 경량 필터를 먼저 삽입**할 수 있을까?

## 4.2 핵심 아이디어: 확률적 술어 (Probabilistic Predicates, PPs)

**핵심 통찰:** 비싼 ML 추론을 수행하기 전에 **경량 이진 분류기를 조기 필터로 사용**하면, 대부분의 음성 사례(false cases)를 미리 제거할 수 있다.

## 오프라인 학습 단계

[Step 1] 비싼 UDF를 소수의 데이터에 실행하여 레이블 수집

- └ 전체 1,000,000 프레임 중
- └ 샘플 1,000 프레임만 선택하여
- └ YOLO 객체 탐지 모델 실행
- └ 레이블 수집 (Red SUV 있음/없음)

[Step 2] 저비용 특징 추출

- └ 색상 히스토그램 (각 RGB 채널의 분포; 512B)
- └ 텍스처 특징 (Edge detection; 256B)
- └ 저해상도 미리보기 (32×32 픽셀; 3KB)
- └ 전체 특징 크기 ~4KB (원본 1080p 프레임 2MB 대비 500배 소)

[Step 3] 경량 이진 분류기 학습

- └ 입력: 1,000개 프레임의 저비용 특징
- └ 출력: Red SUV 있음/없음 (이진)
- └ 알고리즘: 로지스틱 회귀 (선형 모델)
- └ 학습 비용: 1초 미만
- └ 정확도: 95% (비싼 YOLO의 정확도도 97%)

## 온라인 실행 단계

각 프레임에 대해:

[단계 1] 저비용 특징 추출 (4KB)

↓ (비용: 1ms)

[단계 2] 경량 분류기 실행

- └ 99% 확률로 "Red SUV 없음" 판정?
  - ↓ [프레임 스킵; 비싼 YOLO 실행 안 함]
  - ↓ (절약된 비용: 100ms)
- └ 우호적 점수? 계속 진행
  - ↓ (누적 비용: 1ms)

[단계 3] 비싼 YOLO 객체 탐지 실행

↓ (비용: 100ms)

[단계 4] 최종 결과

**극적인 비용 절감:** - 100만 프레임 중 실제 Red SUV: 1,000개 (0.1%) - 경량 필터로 음성 예측 정확도 99%: 990,000개 프레임 제외 - 실제 비싼 YOLO 실행 대상: 10,000개 (9,900개 거짓양성 + 1,000개 실제 양성) - 비용 절감:  $(100만 - 10,000) \times 100ms = 999,000 \times 100ms = \mathbf{99,900초}$  절감

## 4.3 Viola 스타일 캐스케이드 (Cascade of Classifiers)

경량 필터 하나만으로는 부족할 수 있다. 대신 여러 개를 계단식으로 연결한다:

[입력 프레임]

↓

[PP\_1: 초경량 필터 (비용 1ms)]

└ 색상 기반 (빨간색 있는가?)

└ 매우 빠름

└ 정확도는 낮음 (false negative 10%)

└ 90% 제외

↓ [10만 프레임]

[PP\_2: 경량 필터 (비용 5ms)]

└ 색상 + 모양 (직사각형 같은 물체 있는가?)

└ 중간 속도

└ 정확도 중간 (false negative 5%)

└ 80% 추가 제외

↓ [2만 프레임]

[PP\_3: 중간 필터 (비용 30ms)]

└ 에지 감지 + 패턴 매칭 (차량 같은 형태?)

└ 느린 편

└ 정확도 높음 (false negative 2%)

└ 90% 추가 제외

↓ [2,000 프레임]

[비싼 YOLO (비용 100ms)]

└ 정확한 객체 탐지

└ 최종 결과

**캐스케이드의 이점:** 1. 각 단계에서 비용이 점진적으로 증가 (1 → 5 → 30 → 100ms) 2. 각 단계에서 데이터가 걸러짐 (100% → 10% → 2% → 0.2%) 3. 전체 비용 최소화

## 4.4 비용-정확도 트레이드오프 모델링

PP의 정확도-성능 트레이드오프를 비용 기반 옵티마이저에 통합한다:

### 거짓 음성률 (False Negative Rate, FNR)

- 정의: "실제로 Red SUV가 있는데, 필터가 없다고 판단"
- 영향: 최종 결과에서 Red SUV 놓침 (정확도 저하)
- 예; FNR = 5%면 100개 Red SUV 중 5개를 못 찾음

### 거짓 양성률 (False Positive Rate, FPR)

- 정의: "실제로 Red SUV가 없는데, 필터가 있다고 판단"
- 영향: 불필요한 비싼 YOLO 실행 (성능 저하)
- 예; FPR = 10%면 999,000개 음성 중 99,900개가 거짓양성으로 판정되어 비싼 YOLO까지 감

## 옵티마이저의 선택

사용자가 "정확도 손실 1% 이내로, 최대한 빨리" 요청하면:

가능한 PP 조합 평가:

옵션 A: PP 없음 (카스케이드 스킵)

- └ 정확도: 100%
- └ 실행시간: 100,000초 ( $1,000,000 \times 100\text{ms}$ )
- └ 비용 평가: 불합격 (정확도 요구와 무관하게 너무 느림)

옵션 B: PP\_1만 사용

- └ FNR: 10%, FPR: 5%
- └ 최종 정확도: 99.95% (Red SUV 1,000개 중 900개 맞춤)
- └ 실행시간: 1,900 초 ( $1,000,000 \times 1\text{ms} + 50,000 \times 100\text{ms}$ )
- └ 평가: 합격 (정확도 99.95% > 99%, 시간 충분히 단축)

옵션 C: PP\_1 + PP\_2

- └ FNR: 7% (PP\_1 10% 중 PP\_2가 30% 추가 보정)
- └ 최종 정확도: 99.93%
- └ 실행시간: 500초
- └ 평가: 합격 (더욱 빠름)

옵티마이저는 조건을 만족하면서 가장 빠른 옵션 C를 선택한다.

## 4.5 비용 모델의 상세

### 카스케이드 전체 비용

Total Cost = Sum[각 필터별 비용] + Sum[필터 통과한 데이터의 비싼 UDF 비용]

Total =  $(N \times \text{cost}_1) + (N \times k_1 \times \text{cost}_2) + \dots + (N \times k_1 \times k_2 \times \dots \times k_{n-1} \times \text{costUDF})$

여기서:

- $N$  = 전체 입력 데이터 (1,000,000 프레임)
- $\text{cost}_i$  = 필터  $i$ 의 단위 비용 (1ms, 5ms, 30ms)
- $k_i$  = 필터  $i$ 의 통과율 (10%, 20%, 10%)

## 실제 계산 예시

PP\_1 (색상) → PP\_2 (모양) → PP\_3 (패턴) → YOLO

$$\begin{aligned}\text{Total} &= (1,000,000 \times 1\text{ms}) \\ &+ (1,000,000 \times 0.9 \times 5\text{ms}) \\ &+ (900,000 \times 0.8 \times 30\text{ms}) \\ &+ (720,000 \times 0.9 \times 100\text{ms}) \\ &= 1,000\text{초} + 4,500\text{초} + 21,600\text{초} + 64,800\text{초} \\ &= 91,900\text{초 (약 25시간)}\end{aligned}$$

비교:

- 필터 없음: 100,000초 (약 28시간)
- 캐스케이드: 91,900초 (약 26시간)
- 개선: 8% (약 1시간 절감)

→ 여전히 개선의 여지가 있으나, 트레이드오프 고려 필요

## 4.6 성능 결과

이 논문은 SIGMOD 2018 Best Demonstration Award를 수상했으며, 실제 구현 사례들:

### 비디오 프레임 분석

데이터: - WILDTRACK 비디오 데이터셋 (100,000 프레임) - 쿼리: "사람을 탐지하는 프레임" (평균 선택도 30%)

결과:

|              |                    |
|--------------|--------------------|
| 기준 (PP 없음):  | 10,000초 (비싼 YOLO만) |
| PP 캐스케이드 적용: | 100초 (색상 기반 필터)    |
| 성능 향상:       | 100배               |
| 정확도 유지:      | 97% (손실 0%)        |

### 이미지 검색

쿼리: "고양이 탐지" (선택도 5%)

|           |                |
|-----------|----------------|
| 결과:       |                |
| - 기준:     | 150초           |
| - 최적화:    | 5초             |
| - 성능 향상:  | 30배            |
| - 정확도 손실: | 0.1% (허용 범위 내) |

## 4.7 본 논문과의 관계

### 공통점

Exqutor와의 공통 철학: 1. **ML/벡터 연산을 관계형 옵티마이저에 통합** - Exqutor: 벡터 검색의 카디널리티를 정확히 추정 - 이 논문: ML UDF의 비용과 정확도를 모델링

1. **정확한 정보를 옵티마이저에 제공하면 극적 개선 가능**
2. Exqutor: 정확한 선택도 → 최적 조인 순서 선택
3. 이 논문: 정확한 비용 → 최적 필터 캐스케이드 선택
4. **정확도-성능 트레이드오프 관리**
5. AnalyticDB-V: 벡터 검색의 정확도-비용 트레이드오프
6. 이 논문: PP의 FNR-FPR 트레이드오프

### 차이점

Exqutor:

- 대상: 벡터 범위 검색 ( $\text{distance} < D$ )
- 문제: "결과가 정확히 몇 건인가?"
- 해결: HNSW 인덱스 탐색으로 카디널리티 추정

이 논문:

- 대상: 비싼 ML UDF
- 문제: "UDF를 실행할 대상을 최소화하되 정확도는 유지하는가?"
- 해결: 경량 필터 캐스케이드로 불필요한 UDF 제거

### 결합 가능성

두 기법을 함께 사용할 수 있다:

```
-- 벡터 검색 + ML 추론 결합
SELECT * FROM images
WHERE
  -- [1단계] 경량 색상 필터 (PP_1)
  color_histogram_match(image) = TRUE

  AND

  -- [2단계] 벡터 유사도 (Exqutor 대상)
  vector_distance(image_embedding, query_embedding) < 0.5

  AND

  -- [3단계] 비싼 CNN 기반 객체 탐지 (PP_2, PP_3)
  object_detector(image, 'cat') = TRUE;
```

Exqutor가 2단계의 카디널리티를 정확히 예측하면, 옵티마이저가 각 단계의 순서를 최적화할 수 있다.

## 추가 제기 문제

### 1. PP 학습의 적응성:

2. PP를 학습할 때 사용한 1,000개 샘플의 데이터 분포가, 나중의 실제 쿼리와 달라질 수 있다 (Concept Drift)
3. 예; 밤에 찍은 비디오에서는 빨간 SUV의 색상이 검게 보일 수 있음
4. 이 경우 PP의 성능이 떨어진다

**Exqutor의 적응적 샘플링**이 이 문제의 해답이 될 수 있을까? - 런타임에 PP의 오류율을 모니터링 - 오류율이 높아지면 새 데이터로 PP를 재학습

### 1. 여러 쿼리 간 PP 공유:

2. 현재는 각 쿼리마다 별도의 PP 캐스케이드를 구성
3. 예; "빨간 차", "파란 차", "검은 차" 쿼리가 모두 색상 필터 공유 가능
4. 이런 공유를 자동 탐지하는 메커니즘?

### 5. PP의 학습 비용 고려:

6. 1,000개 샘플에 대해 비싼 YOLO 실행 (100초)
7. 분류기 학습 (1초)
8. 이 초기 비용이 언제쯤 회수되는가?
9. 만약 쿼리가 few-shot (한 번만 실행)이라면 학습 비용이 낭비됨

### 10. 비선형 필터 조합:

11. 현재는 선형 캐스케이드 (PP\_1 → PP\_2 → PP\_3)



12. 하지만 일부 필터는 병렬 실행 가능한가? (예; 색상 필터와 에지 필터 동시 실행)
13. 이런 병렬화로 추가 최적화 가능?